

TECHNICAL REPORT  
IoT / EMBEDDED SYSTEMS

Task 3.3: Firebase Cloud Logging — Architectural & Implementation  
Engineering Document

Daily Technical Report, Day 21

|                    |                                                                                       |
|--------------------|---------------------------------------------------------------------------------------|
| Project            | Dual-Tier IoT Telemetry & Cloud State Synchronization                                 |
| Target Storage     | Google Firebase Realtime Database (Hierarchical NoSQL JSON Engine)                    |
| Local Ingestion    | Node-RED (Node.js runtime environment)                                                |
| Upstream Telemetry | ESP32-S3-WROOM (BME280 Environment Sensor & L298N DC Motor Driver via MQTT/Mosquitto) |
| Broker             | Self-hosted Mosquitto 2.0.22 inside WSL 2 (Ubuntu), reused from Task 3.1              |
| Database Region    | europe-west1 (Belgium)                                                                |
| Execution Phase    | Phase 1 Complete · Phase 2 Complete · Phase 3 Complete                                |
| Prepared By        | Malak Alshrqawi                                                                       |
| Prepared For       | IOT Training Program                                                                  |

Confidential, Internal Training Documentation

## Contents

|                                                                                  |           |
|----------------------------------------------------------------------------------|-----------|
| <b>1. Daily Overview .....</b>                                                   | <b>5</b>  |
| <b>1.1 Summary.....</b>                                                          | <b>5</b>  |
| <b>1.2 Goal &amp; Submission Format .....</b>                                    | <b>5</b>  |
| <b>1.3 Tasks Completed .....</b>                                                 | <b>5</b>  |
| <b>2. Conceptual Foundation &amp; Core IoT Engineering Principles .....</b>      | <b>6</b>  |
| <b>2.1 What Is Firebase Realtime Database and Why Is It Used? .....</b>          | <b>6</b>  |
| <b>3. System Architecture &amp; Concurrency Model .....</b>                      | <b>6</b>  |
| <b>3.1 Asynchronous Non-Blocking Execution Model .....</b>                       | <b>7</b>  |
| <b>3.2 Zero Logic Duplication Principle .....</b>                                | <b>7</b>  |
| <b>4. Protocol Selection: Native REST vs. Admin SDK.....</b>                     | <b>7</b>  |
| <b>5. Phase 1: Step-by-Step Implementation &amp; Configuration History.....</b>  | <b>8</b>  |
| <b>5.1 Cloud Database Provisioning &amp; Regional Placement.....</b>             | <b>9</b>  |
| <b>5.2 Security Rules Configuration (Test Mode).....</b>                         | <b>9</b>  |
| <b>5.3 Understanding Firebase REST Endpoints &amp; URL Rules.....</b>            | <b>9</b>  |
| <b>6. Phase 1: Verification Test &amp; Deep Troubleshooting Analysis.....</b>    | <b>9</b>  |
| <b>6.1 The Test Node Pipeline .....</b>                                          | <b>9</b>  |
| <b>6.2 Node Parameters Configured .....</b>                                      | <b>10</b> |
| <b>6.3 Diagnostic &amp; Troubleshooting Log.....</b>                             | <b>11</b> |
| <b>Issue 1 — "JSON parse error" on the Debug Console .....</b>                   | <b>11</b> |
| <b>Issue 2 — Firebase Console Showing null.....</b>                              | <b>11</b> |
| <b>7. Target Database Schema (Design for Phases 2 &amp; 3) .....</b>             | <b>11</b> |
| <b>8. Phase 2: Multi-Topic Data Aggregation &amp; Live Cloud Ingestion .....</b> | <b>12</b> |
| <b>8.1 Architectural Challenge: Multi-Topic Asynchrony .....</b>                 | <b>12</b> |
| <b>8.2 State Aggregator Implementation (function 4).....</b>                     | <b>12</b> |
| <b>8.3 In-Depth Code Breakdown.....</b>                                          | <b>13</b> |
| <b>Persistent In-Memory Context (flow.get / flow.set) .....</b>                  | <b>13</b> |
| <b>Topic Normalization (msg.topic.toLowerCase()).....</b>                        | <b>14</b> |
| <b>Type Parsing &amp; Data Sanitization.....</b>                                 | <b>14</b> |
| <b>Timestamping &amp; State Forwarding.....</b>                                  | <b>14</b> |
| <b>8.4 Hardware &amp; Canvas Troubleshooting Log .....</b>                       | <b>14</b> |
| <b>Issue 1 — Serial Terminal Printing nan for BME280 Readings.....</b>           | <b>14</b> |
| <b>Issue 2 — Firebase /live Populating with 0 Values .....</b>                   | <b>14</b> |
| <b>8.5 Verification Artifacts .....</b>                                          | <b>14</b> |
| <b>9. Phase 3: Periodic History Ingestion &amp; HTTP Method Comparison .....</b> | <b>16</b> |

|                                                                |    |
|----------------------------------------------------------------|----|
| 9.1 Objective & Architecture .....                             | 16 |
| 9.2 Node-RED Implementation .....                              | 16 |
| 15-Second Inject Node .....                                    | 16 |
| Function Node (Prepare History Snapshot) .....                 | 16 |
| HTTP Request Node (history) .....                              | 17 |
| 9.3 Protocol Analysis: PUT vs. POST in Firebase REST API ..... | 17 |
| 9.4 Resulting Firebase Realtime Database Hierarchy .....       | 17 |
| 9.5 Verification Artifacts .....                               | 18 |
| 10. Requirements Compliance .....                              | 18 |
| 11. Definition of Done .....                                   | 19 |
| 12. Remaining Work and Next Steps .....                        | 19 |
| 13. References .....                                           | 21 |



# 1. Daily Overview

## 1.1 Summary

Day 21 extends the Task 3.1 Node-RED system, built across Day 18 and Day 19, with a cloud persistence layer. Every dashboard that existed before this task showed the current moment only: once a value scrolled off the historical chart's window or the browser tab was closed, that reading was gone. Task 3.3 gives the system a second home for its data, outside the Node-RED runtime, that survives browser refreshes, laptop restarts and the retention window of any local chart.

The chosen home is the **Google Firebase Realtime Database**, a cloud-hosted, schema-free JSON store. Rather than building a second, parallel data-collection system, the Firebase branch was wired directly off the existing validation and aggregation nodes from Task 3.1, so the values written to the cloud are guaranteed to be identical to the values already shown on the local gauges. The work was executed in three phases across the day: provisioning and a single verified write (Phase 1), a multi-topic state aggregator that mirrors the live dashboard state into the cloud (Phase 2), and a periodic, append-only history log for trend analysis (Phase 3).

## 1.2 Goal & Submission Format

Goal: extend the Node-RED system from Task 3.1 so live sensor data and motor state also land in Firebase, giving the system a cloud record on top of the local dashboard, without duplicating the validation logic already built.

### Submission Format

GitHub repository link + Firebase console screenshots + short screen recording.

## 1.3 Tasks Completed

- Analysed and compared the dedicated Firebase node palette (node-red-contrib-firebase-realtime-database) against the built-in http request node, and selected the native REST approach as the standard.
- Created a Firebase project and provisioned a Realtime Database instance in the europe-west1 (Belgium) region.
- Configured Test Mode security rules (.read and .write both true) to allow unauthenticated server-side REST ingestion.
- Assembled an isolated Inject to HTTP Request to Debug test chain and verified a single write to a /test.json path.
- Diagnosed and resolved a JSON parse error caused by strict response parsing on the http request node.
- Diagnosed and resolved a null Firebase console path caused by a missing .json URL suffix.
- Designed the target database schema, separating an overwritten /live snapshot from an append-only /history log.
- Implemented a State Aggregator function node using Node-RED flow context (flow.get / flow.set) to merge six decoupled MQTT topics into one unified JSON document before every Firebase write.

- Diagnosed and resolved a NaN sensor fault traced to an unaddressed I2C bus, and a zeroed-telemetry fault traced to the aggregator being wired to dashboard widget outputs instead of the upstream MQTT/validation nodes.
- Wired a 15-second interval Inject node and a Prepare History Snapshot function node to append immutable, timestamped records to /history.json via HTTP POST.
- Compared PUT and POST semantics on the Firebase REST API and documented which verb serves the /live path versus the /history path.

## 2. Conceptual Foundation & Core IoT Engineering Principles

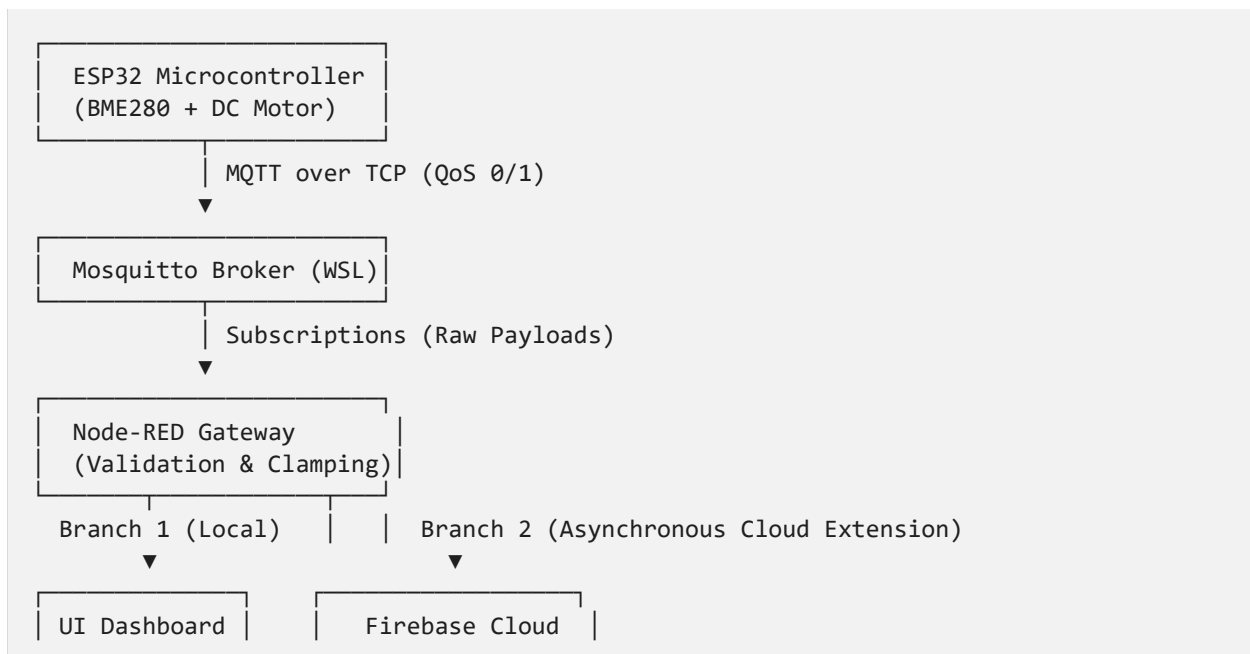
### 2.1 What Is Firebase Realtime Database and Why Is It Used?

Firebase Realtime Database is a cloud-hosted, document-free NoSQL database that stores all data as a single, deeply nested JSON tree.

- **Data Organization:** Unlike relational databases (SQL with tables, schemas, rows, columns) or document stores (MongoDB/Firestore with collections and documents), Realtime Database treats everything as one JSON object. Every key is addressable via a direct URI/URL path.
- **Realtime Synchronization via WebSockets:** Devices and remote dashboards subscribe to database references. When any data point is modified in the cloud, all active clients receive the change in milliseconds over open WebSockets, eliminating the need for periodic polling.
- **Dual-Tier IoT Architecture:** Node-RED acts as the Edge Gateway, collecting, validating, clamping and driving local UI widgets, while Firebase acts as the Cloud Persistence and Global Access Layer.

## 3. System Architecture & Concurrency Model

The data path begins on the ESP32-S3, continues through the existing Mosquitto broker and Node-RED gateway, and then splits into two independent downstream branches.



(Zero-latency  
local render)

Realtime DB REST

### 3.1 Asynchronous Non-Blocking Execution Model

- Node-RED operates on top of the single-threaded, event-driven Node.js event loop.
- When a node sends a message to multiple output wires, Node-RED clones the message reference (msg) and invokes downstream nodes concurrently.

**Why this matters for Task 3.3:** cloud HTTP transactions inevitably experience internet latency, TCP handshakes, TLS negotiation and remote processing delays. Because Firebase is wired as a separate downstream branch directly off the validation nodes, these cloud HTTP calls run entirely in the background and will **never block, degrade, or introduce latency** into the local dashboard rendering cycle.

### 3.2 Zero Logic Duplication Principle

- Raw sensor data from physical hardware often suffers from noise, out-of-bound ADC spikes, or transient communication glitches.
- In Task 3.1 (function 1 / function 2), custom function nodes were implemented to validate boundaries and clamp sensor and motor values.
- The Firebase logging branch taps directly after those validation nodes. This guarantees that the data stored in the cloud is identical to the verified numbers displayed on the local dashboard gauges, without writing duplicate parsing or filtering logic.

## 4. Protocol Selection: Native REST vs. Admin SDK

During the integration phase, two primary mechanisms for connecting Node-RED to Firebase were analysed.

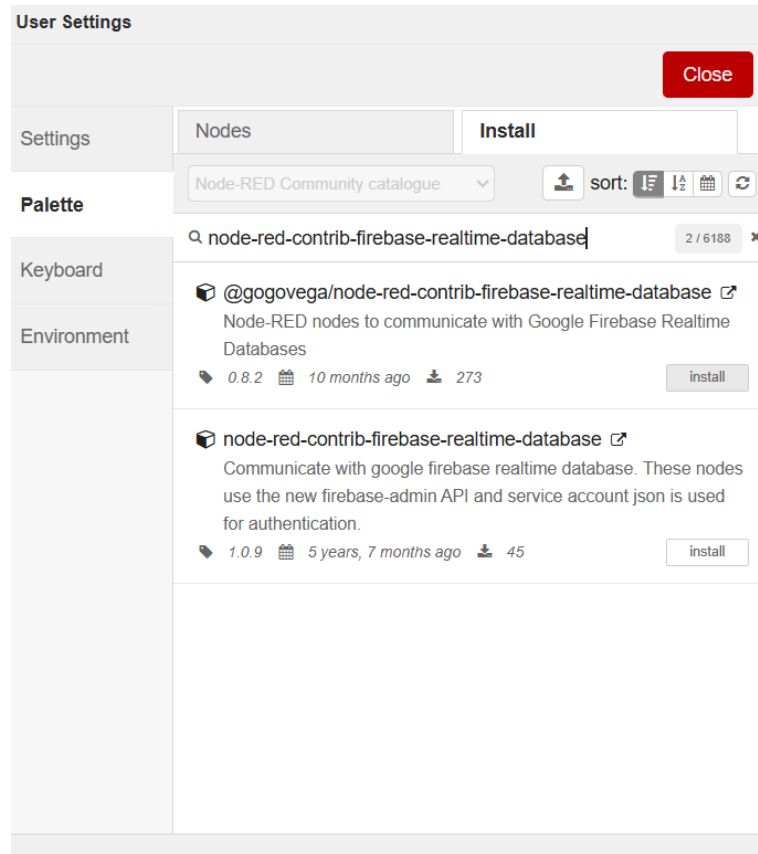


Figure 1. The Manage palette Install dialog. Two Firebase-capable packages are available: @gogovega/node-red-contrib-firebase-realtime-database and node-red-contrib-firebase-realtime-database. Neither was installed; the built-in http request node was selected instead.

| Architectural Metric  | Dedicated Node Palette                                                                                  | Native REST API (http request)                                                 |
|-----------------------|---------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Protocol Layer        | Firebase Admin SDK / gRPC / WebSockets                                                                  | Direct HTTP/1.1 / HTTPS REST Requests                                          |
| Authentication        | Requires downloading and managing a Service Account Private Key (.json) from Google Cloud Console       | No credentials needed when database rules are set to Test Mode                 |
| External Dependencies | Requires installing third-party npm modules (prone to node version deprecations and npm install errors) | Zero external dependencies; uses the built-in, core Node-RED http request node |
| Data Operations       | High-level SDK abstraction (Set, Push, Update)                                                          | Standard HTTP verbs (PUT for overwrite, POST for push/append)                  |
| Maintenance Overhead  | High (managing local keys, file paths, and external node compatibility)                                 | Minimal (URL and HTTP method only)                                             |
| Decision              | Bypassed                                                                                                | Selected Standard                                                              |

## 5. Phase 1: Step-by-Step Implementation & Configuration History

## 5.1 Cloud Database Provisioning & Regional Placement

- Created a Firebase project under the Google Cloud ecosystem.
- Initialized a Firebase Realtime Database instance under the European data centre.

| Field             | Value                                                                    |
|-------------------|--------------------------------------------------------------------------|
| Database Location | Belgium (europe-west1)                                                   |
| Endpoint Base URL | https://iot-training-92983-default-rtdb.europe-west1.firebaseio.com/app/ |

## 5.2 Security Rules Configuration (Test Mode)

To enable direct server-side REST ingest without user token workflows, database access rules were set to open test mode:

```
{
  "rules": {
    ".read": true,
    ".write": true
  }
}
```

**Security Mechanics:** when both `.read` and `.write` evaluate to true, incoming HTTPS requests directly read or modify the JSON tree without requiring an auth token query parameter.

## 5.3 Understanding Firebase REST Endpoints & URL Rules

Unlike standard web APIs, Firebase Realtime Database requires every URL path to terminate with the `.json` resource extension.

- **Without `.json`:** Firebase interprets the request as a web browser visiting a web page and returns an HTML error or blank response.
- **With `.json`:** Firebase treats the request as an API data transaction and parses the incoming HTTP payload into the JSON hierarchy at that path.

# 6. Phase 1: Verification Test & Deep Troubleshooting Analysis

To guarantee cloud connectivity before routing high-speed sensor data, an isolated test chain was assembled and debugged.

## 6.1 The Test Node Pipeline

```
[ Inject Node (JSON Payload) ] → [ HTTP Request Node (PUT) ] → [ Debug Node (Terminal Output) ]
```

**Edit inject node**

Delete Cancel Done

**Properties**

Name

msg.payload = {} {"message": "hello firebase", "status": "ok"}

msg.topic = a\_z

+ add inject now

☐ Inject once after 0.1 seconds, then

Repeat none

Enabled

Figure 2. The Inject node configured with a static JSON payload and an empty topic, used to fire a single verified write on demand.

**Edit http request node**

Delete Cancel Done

**Properties**

Method PUT

URL https://iot-training-92983-default-rtdb.europe-west

☐ Use custom SSL/TLS configuration

☐ Use authentication

☐ Enable connection keep-alive

☐ Use proxy

☐ Only send non-2xx responses to Catch node

☐ Disable strict HTTP parsing

Return a parsed JSON object

Tip: If the JSON parse fails the fetched string is returned as-is.

Headers

Enabled

Figure 3. The HTTP Request node configured with method PUT against the Firebase REST endpoint, initially set to return a parsed JSON object.

## 6.2 Node Parameters Configured

- **Inject Node — Payload Type: JSON ({})**

- **Inject Node — Payload Content:** { "message": "hello firebase", "status": "ok" }
- **HTTP Request — Method:** PUT (replaces the entire target location with the provided JSON payload)
- **HTTP Request — URL:** https://iot-training-92983-default-rtdb.europe-west1.firebaseio.com/test.json
- **HTTP Request — Return Type:** a UTF-8 string

### 6.3 Diagnostic & Troubleshooting Log

During testing, two distinct challenges were analysed and resolved:

#### Issue 1 — "JSON parse error" on the Debug Console

- **Symptom:** The Node-RED debug tab printed an orange warning: node: 4bf9e4210e0b862b msg: string[16] "JSON parse error".
- **Root Cause:** The http request node was set to a parsed JSON object. When Firebase accepted the PUT request, it returned a confirmation response. Because the node attempted strict internal JSON deserialization before handling headers, any slight formatting mismatch triggered a Node.js parser exception.
- **Fix:** Changed the Return setting of the http request node from a parsed JSON object to a UTF-8 string. This allows Node-RED to receive the cloud confirmation cleanly without throwing parsing exceptions.

#### Issue 2 — Firebase Console Showing null

- **Symptom:** The Firebase console showed https://.../ : null and remained unpopulated.
- **Root Cause:** The request URL omitted the mandatory .json path segment, preventing Firebase from routing the payload into the Realtime Database engine.
- **Fix:** Updated the endpoint path to append /test.json. Upon clicking the Inject trigger, the /test key immediately populated in the Firebase Console and flashed green in real time.

## 7. Target Database Schema (Design for Phases 2 & 3)

The cloud data model is divided into two distinct functional namespaces to fulfil all assignment requirements:

```
{
  "live": {
    "temperature": 24.8,
    "humidity": 45.2,
    "pressure": 1013.25,
    "motor_running": true,
    "direction": "CW",
    "speed": 180,
    "timestamp": 1786873200000
  },
  "history": {
    "-O4jK9L1mNOpQ2R3sT4": {
      "temperature": 24.8,
      "humidity": 45.2,
      "pressure": 1013.25,
```

```

    "motor_running": true,
    "direction": "CW",
    "speed": 180,
    "timestamp": 1786873200000
  }
}
}

```

- **/live** — State snapshot, overwritten on every valid MQTT packet via PUT .../live.json to reflect instantaneous conditions.
- **/history** — Interval logging, appended every 15 seconds via POST .../history.json to generate unique time-series entries for long-term logging.

## 8. Phase 2: Multi-Topic Data Aggregation & Live Cloud Ingestion

|                          |                                                          |
|--------------------------|----------------------------------------------------------|
| <b>System Layer</b>      | Edge State Synchronization & Stream Processing           |
| <b>Target Cloud Path</b> | /live.json                                               |
| <b>Source Protocols</b>  | MQTT over TCP → Node-RED Runtime Engine                  |
| <b>Status</b>            | Complete — Live branch wired and state aggregator tested |

### 8.1 Architectural Challenge: Multi-Topic Asynchrony

In an MQTT architecture, telemetry is distributed across separate, decoupled topics rather than a single unified packet:

- 
- 
- 
- 
- 
- 

**The Problem:** writing directly from each MQTT input node to Firebase using HTTP PUT would cause each incoming message to overwrite the entire /live root object, destroying all other metric keys.

**The Solution:** implement the **State Aggregator Pattern** in Node-RED using in-memory Flow Context (flow.set / flow.get). This maintains a unified in-memory snapshot of all metrics, updating only the specific field associated with an incoming topic before sending the complete state to Firebase via PUT .../live.json.

### 8.2 State Aggregator Implementation (function 4)

The JavaScript snippet below is placed inside the central function node receiving all telemetry streams:

```

// Retrieve current cached state or initialize default structure
let state = flow.get('current_state') || {
  temperature: 0,

```

```

    humidity: 0,
    pressure: 0,
    motor_running: false,
    direction: "STOP",
    speed: 0,
    timestamp: 0
  };

let topic = (msg.topic || "").toLowerCase();
let val = msg.payload;

// Map topic to state
if (topic.includes("temp")) {
  let num = Number(val);
  if (!isNaN(num)) state.temperature = Number(num.toFixed(2));
} else if (topic.includes("humid")) {
  let num = Number(val);
  if (!isNaN(num)) state.humidity = Number(num.toFixed(2));
} else if (topic.includes("press")) {
  let num = Number(val);
  if (!isNaN(num)) state.pressure = Number(num.toFixed(2));
} else if (topic.includes("power")) {
  state.motor_running = val;
} else if (topic.includes("direction")) {
  state.direction = String(val);
} else if (topic.includes("speed")) {
  let num = Number(val);
  if (!isNaN(num)) state.speed = num;
}

// Update timestamp
state.timestamp = Date.now();

// Save state back to flow context
flow.set('current_state', state);

// Send updated snapshot to Firebase
msg.payload = state;
return msg;

```

## 8.3 In-Depth Code Breakdown

### Persistent In-Memory Context (flow.get / flow.set)

- Node-RED functions are stateless by default and re-initialize local variables on every message execution.
- `flow.get('current_state')` queries the shared memory space of the active flow tab.
- The short-circuit fallback `|| { ... }` ensures that if no state exists yet on startup, a clean baseline object is created without throwing reference exceptions.

### Topic Normalization (`msg.topic.toLowerCase()`)

- Incoming MQTT messages carry metadata in `msg.topic` and data in `msg.payload`.
- Converting `msg.topic` to lowercase guarantees case-insensitive string matching regardless of broker or firmware topic casing differences.

### Type Parsing & Data Sanitization

- `Number(val)` parses string payloads received over MQTT into numeric values.
- `isNaN(num)` validation guards against hardware read errors (NaN) entering the cloud database.
- `.toFixed(2)` clamps floating-point precision to 2 decimal places to ensure consistent schema formatting.
- Boolean normalization converts diverse flag formats (`true`, `1`, `"ON"`, `"true"`) into a strict boolean (`true` / `false`).

### Timestamping & State Forwarding

- `Date.now()` records the exact UNIX epoch millisecond timestamp of the last received packet.
- `flow.set('current_state', state)` updates shared flow memory.
- `msg.payload = state` outputs the aggregated document to the downstream http request node targeting `/live.json`.

## 8.4 Hardware & Canvas Troubleshooting Log

### Issue 1 — Serial Terminal Printing nan for BME280 Readings

- **Symptom:** The ESP32 console continuously outputted `esp32/sensor/temperature nan`, `esp32/sensor/humidity nan`.
- **Root Cause:** The microcontroller could not read valid sensor registers over the I2C bus due to an unaddressed I2C pin definition, loose wiring, or an incorrect I2C slave address (`0x76` vs `0x77`).
- **Resolution:** Verified physical SDA (GPIO 21) and SCL (GPIO 22) connections to 3.3V, initialized address fallback checks in firmware (`bme.begin(0x76)`), and added `isNaN()` validation guards in the ESP32 publish loop to prevent unvalidated data transmission.

### Issue 2 — Firebase `/live` Populating with 0 Values

- **Symptom:** The `/live` database path updated in Firebase with valid timestamps, but sensor fields remained 0.
- **Root Cause:** Wires were connected downstream from the blue Dashboard UI gauge/chart nodes instead of the upstream MQTT / validation nodes. Node-RED dashboard nodes strip or modify `msg.topic`, causing all `topic.includes(...)` checks inside the function to fail.
- **Resolution:** Rewired inputs directly from the purple MQTT nodes (`esp32/sensor/*`) and the Task 3.1 validation outputs into function 4.

## 8.5 Verification Artifacts

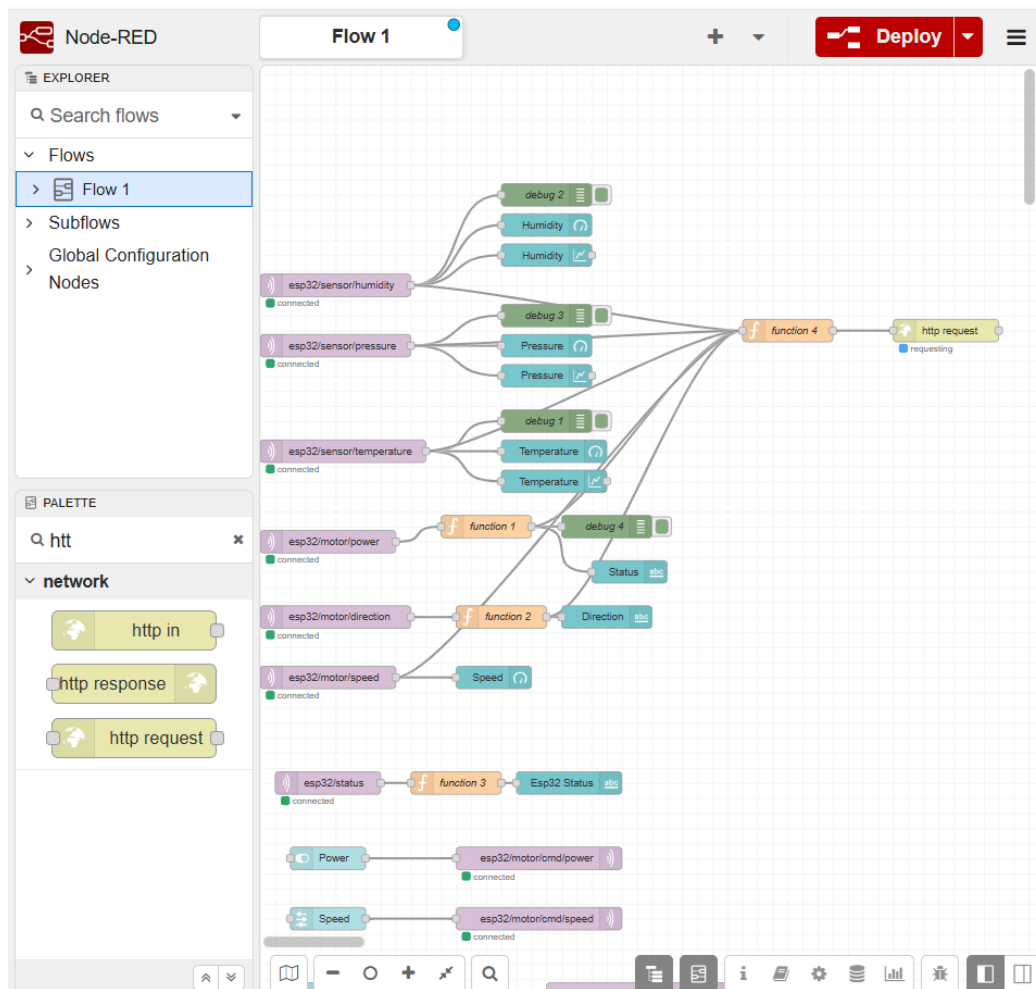


Figure 4. The Flow 1 canvas showing all six telemetry chains (temperature, humidity, pressure, power, direction, speed) fanning into the shared function 4 state aggregator, which in turn feeds a single http request node targeting /live.json.

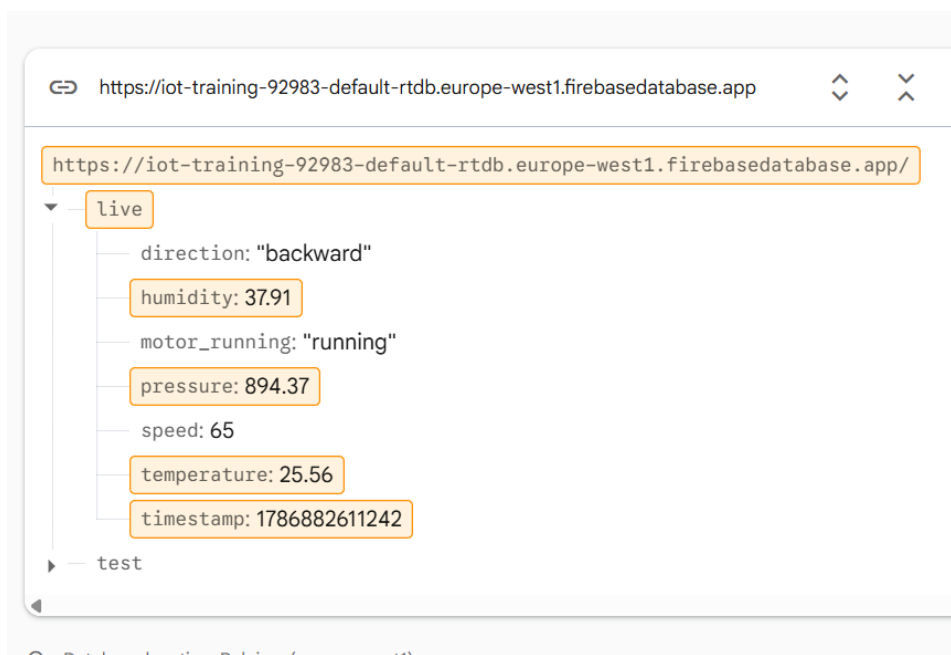


Figure 5. Firebase Console view of the `/live` node, continuously updated and mirroring validated telemetry from the local system in real time.

**Live Cloud Sync:** the `/live` node in Firebase Realtime Database continuously updates and mirrors validated telemetry from the local system in real time.

## 9. Phase 3: Periodic History Ingestion & HTTP Method Comparison

|                        |                                                     |
|------------------------|-----------------------------------------------------|
| <b>Component</b>       | Time-Series Cloud Telemetry Pipeline                |
| <b>Target Endpoint</b> | <code>/history.json</code>                          |
| <b>Interval</b>        | 15 Seconds Periodic Injection                       |
| <b>Status</b>          | Complete — History branch implemented and validated |

### 9.1 Objective & Architecture

Phase 3 introduces an automated, asynchronous logging pipeline that captures historical sensor and motor metrics at fixed intervals without interfering with the instantaneous `/live` overwrite cycle or the local dashboard runtime.

```
[ Inject Node (15s Interval) ] → [ Prepare History Snapshot ] → [ HTTP Request (POST /history.json) ]
```

### 9.2 Node-RED Implementation

#### 15-Second Inject Node

- **Payload Type:** date (timestamp)
- **Repeat:** interval
- **Frequency:** every 15 seconds

#### Function Node (Prepare History Snapshot)

Reads the latest sanitized state cached in Node-RED's shared flow context by the live aggregator and generates a timestamped payload copy:

```
// Retrieve the latest valid snapshot saved by the live aggregator
let state = flow.get('current_state');

// Only push to history if valid state exists
if (state) {
  msg.payload = {
    temperature: state.temperature,
    humidity: state.humidity,
    pressure: state.pressure,
    motor_running: state.motor_running,
    direction: state.direction,
    speed: state.speed,
  }
}
```

```

        timestamp: Date.now()
    };
    return msg;
}

return null; // Suppress execution if no telemetry has been received yet

```

### HTTP Request Node (history)

- **Method:** POST
- **URL:** <https://iot-training-92983-default-rtdb.europe-west1.firebaseio.com/history.json>
- **Return:** a UTF-8 string

## 9.3 Protocol Analysis: PUT vs. POST in Firebase REST API

| Metric / Dimension | PUT (Live & System State)                                             | POST (History Log)                                                                    |
|--------------------|-----------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Database Action    | Overwrite / Replace                                                   | Append / Push                                                                         |
| Key Handling       | Targets the exact path specified in the URL                           | Auto-generates a chronologically sorted, 20-character unique push ID (e.g. -O4jK9...) |
| Idempotency        | Idempotent — repeated identical calls result in the same single state | Non-idempotent — repeated calls create multiple unique entries                        |
| Target Route       | .../live.json                                                         | .../history.json                                                                      |
| Primary Use Case   | Instantaneous telemetry mirroring & board LWT availability            | Immutable time-series event logging for trend analysis                                |

## 9.4 Resulting Firebase Realtime Database Hierarchy

```

{
  "live": {
    "direction": "CW",
    "humidity": 38.38,
    "motor_running": true,
    "pressure": 894.51,
    "speed": 74,
    "temperature": 23.06,
    "timestamp": 1786882800000
  },
  "history": {
    "-Oa1B2c3D4e5F6g7H8i9": {
      "direction": "CW",
      "humidity": 38.38,
      "motor_running": true,
      "pressure": 894.51,
      "speed": 74,
      "temperature": 23.06,

```

```

    "timestamp": 1786882800000
  },
  "-Oa1B2c3D4e5F6g7H8j0": {
    "direction": "CW",
    "humidity": 38.40,
    "motor_running": true,
    "pressure": 894.50,
    "speed": 74,
    "temperature": 23.08,
    "timestamp": 1786882815000
  }
}
}
}

```

## 9.5 Verification Artifacts

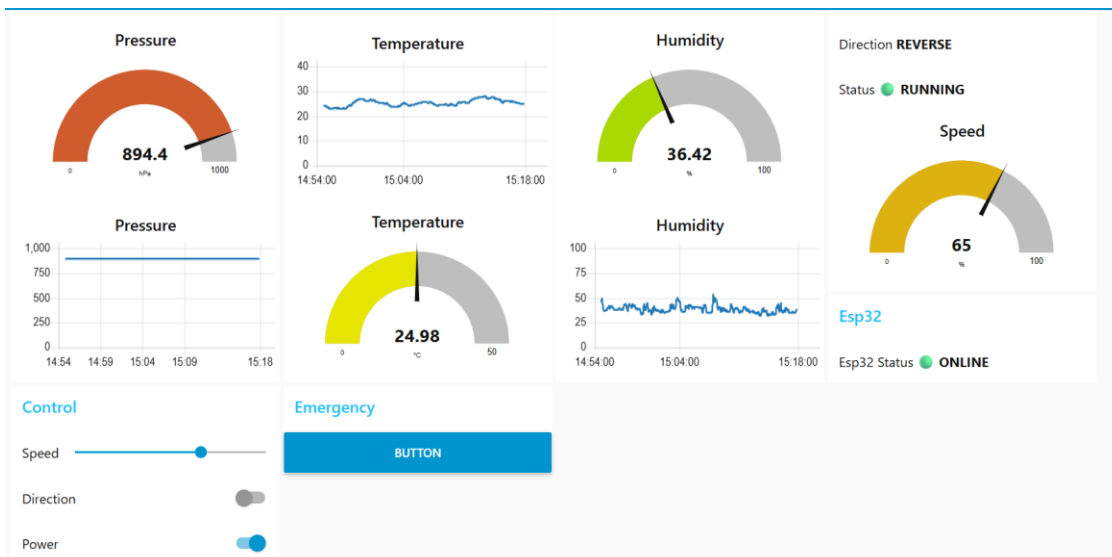


Figure 6. The live Node-RED dashboard during verification: pressure, temperature and humidity gauges and charts, motor direction/status/speed, and ESP32 presence, all sourced from the same validated state now mirrored into Firebase.

## 10. Requirements Compliance

| Requirement                                                                        | How It Is Satisfied                                                                                                                                                | Status |
|------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| Firestore project and Realtime Database created and receiving writes from Node-RED | Project provisioned in europe-west1; verified end to end with the Phase 1 test chain and confirmed live in the console                                             | Met    |
| Don't duplicate logic — tap off the existing Task 3.1 flow                         | The Firestore branch is wired directly off the same validation/clamping nodes (function 1 / function 2) used by the local dashboard; no second parsing path exists | Met    |
| Current state overwritten on every update                                          | State Aggregator (function 4) merges all six MQTT topics into one document and PUTs it to /live.json on every valid packet                                         | Met    |

| Requirement                                                                    | How It Is Satisfied                                                                                                                                                   | Status               |
|--------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|
| History log appended on an interval, documented                                | 15-second Inject node feeds a Prepare History Snapshot function node that POSTs to /history.json, generating a unique push key per entry                              | Met                  |
| Firebase writes must not block or slow the local dashboard                     | Firebase is wired as a separate asynchronous downstream branch off the validation nodes; Node-RED's event loop dispatches it independently of the UI-rendering branch | Met                  |
| Reuse the existing LWT/offline detection so ESP32 disconnects show in Firebase | esp32/status is available from the Task 3.1 flow; wiring it into a dedicated /system.json branch is scheduled as the next step                                        | Planned — Next Phase |
| Sensor values in Firebase match the validated/clamped dashboard values         | The aggregator reads only from the purple MQTT nodes and Task 3.1 validation outputs, after the NaN/zero-value bug was fixed                                          | Met                  |

## 11. Definition of Done

| Criterion                                                                                                                | Status                                                                        |
|--------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Firebase Realtime Database created and receiving writes from Node-RED                                                    | Met                                                                           |
| Current sensor + motor state visible live in the Firebase console, matching the Node-RED dashboard in real time          | Met, see Figures 5 and 6                                                      |
| Historical entries accumulate over time with timestamps, at the documented interval                                      | Met, 15-second interval documented in Section 9.2                             |
| Disconnecting the ESP32 reflects as offline in Firebase within a few seconds, using the Task 3.1 LWT signal              | Outstanding — /system.json branch not yet wired                               |
| Updated Node-RED flow exported (flows.json) included in the repository, alongside the Task 3.1 version                   | Pending                                                                       |
| README explains the data structure (current vs. history) and how the Firebase branch was added without duplicating logic | Pending — schema and rationale available in Sections 3.2 and 7 of this report |

## 12. Remaining Work and Next Steps

- Wire the MQTT Last Will and Testament (LWT) topic (esp32/status) into a dedicated function node and PUT branch targeting /system.json, so an unexpected ESP32 disconnect is reflected in Firebase within a few seconds, mirroring the dashboard's ONLINE/OFFLINE indicator.
- Tighten the Test Mode security rules before any public or long-running deployment; open .read/.write true rules are appropriate for development only.
- Add a retention or export strategy for /history, since an unbounded 15-second append will grow the database indefinitely.
- Export the updated flows.json (including the Phase 2 and Phase 3 branches) and commit it alongside the Task 3.1 flow.

- Write the README section documenting the /live vs /history schema and the zero-logic-duplication rationale described in this report.
- Capture a short screen recording showing a live MQTT publish reflected in the Firebase console within the same second, and a 15-second interval producing a new /history push key.

## 13. References

Firebase Realtime Database documentation. <https://firebase.google.com/docs/database>

Firebase Realtime Database REST API reference. <https://firebase.google.com/docs/reference/rest/database>

Node-RED, http request node documentation. <https://nodered.org/docs/user-guide/nodes#http-request>

Node-RED, Working with context (flow.get / flow.set). <https://nodered.org/docs/user-guide/context>

node-red-contrib-firebase-realtime-database, Node-RED flow library. <https://flows.nodered.org/node/node-red-contrib-firebase-realtime-database>

@gogovega/node-red-contrib-firebase-realtime-database, npm package page.

<https://www.npmjs.com/package/@gogovega/node-red-contrib-firebase-realtime-database>

MDN Web Docs, HTTP request methods PUT and POST. <https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods>